Predicting transition state chemical structures with generative models

Siddarth Achar

AI Schmidt Fellow

04/14/2026

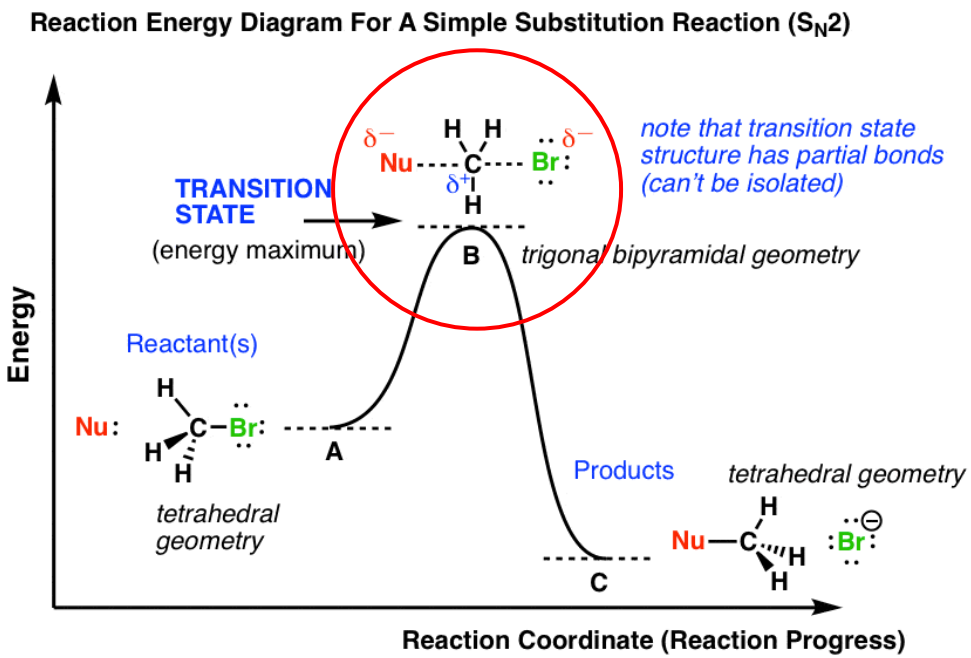
THE UNIVERSITY OF CHICAGO DATA SCIENCE INSTITUTE

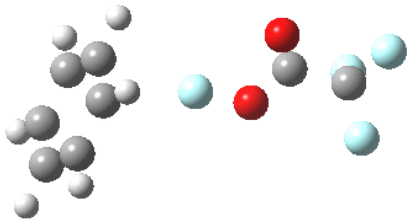
Schmidt Sciences

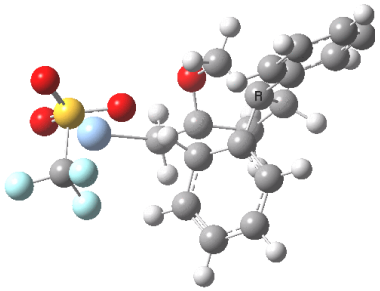
THE UNIVERSITY OF CHICAGO

What about reaction pathways?

- Transition states (TS) control reaction rates and mechanisms
- Computing TSs is very expensive
- We can reduce cost across reactions





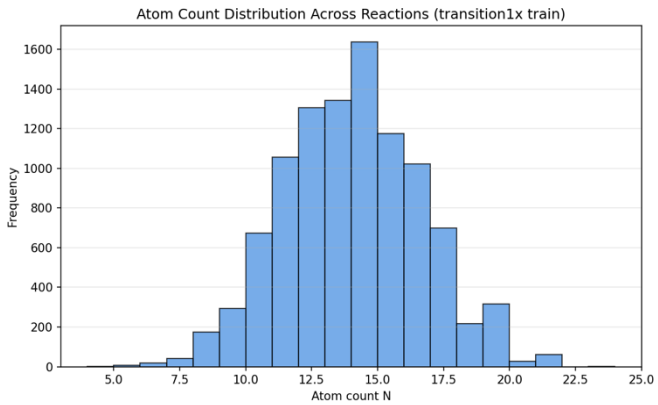


Aromatic electrophilic substitution as part of a catalytic process

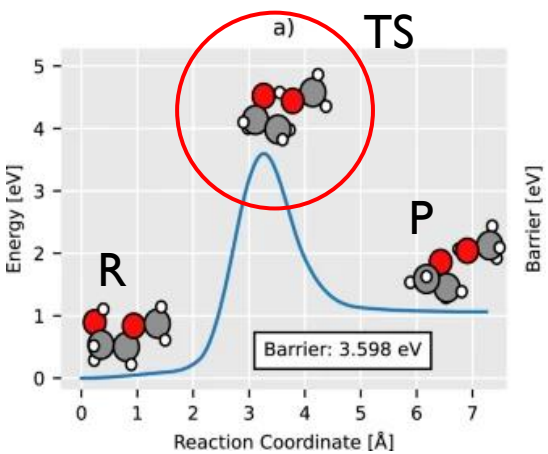
Challenge: can we predict accurate TS geometries of chemical reactions with a generalized model?

Transition1x dataset

- ~10,000 reaction geometries (Transition1x)
- Atom-mapped reactants, products, and transition states
- All geometries provided as Cartesian coordinates
- Energies and forces available (density functional theory (quantum mechanics) level)

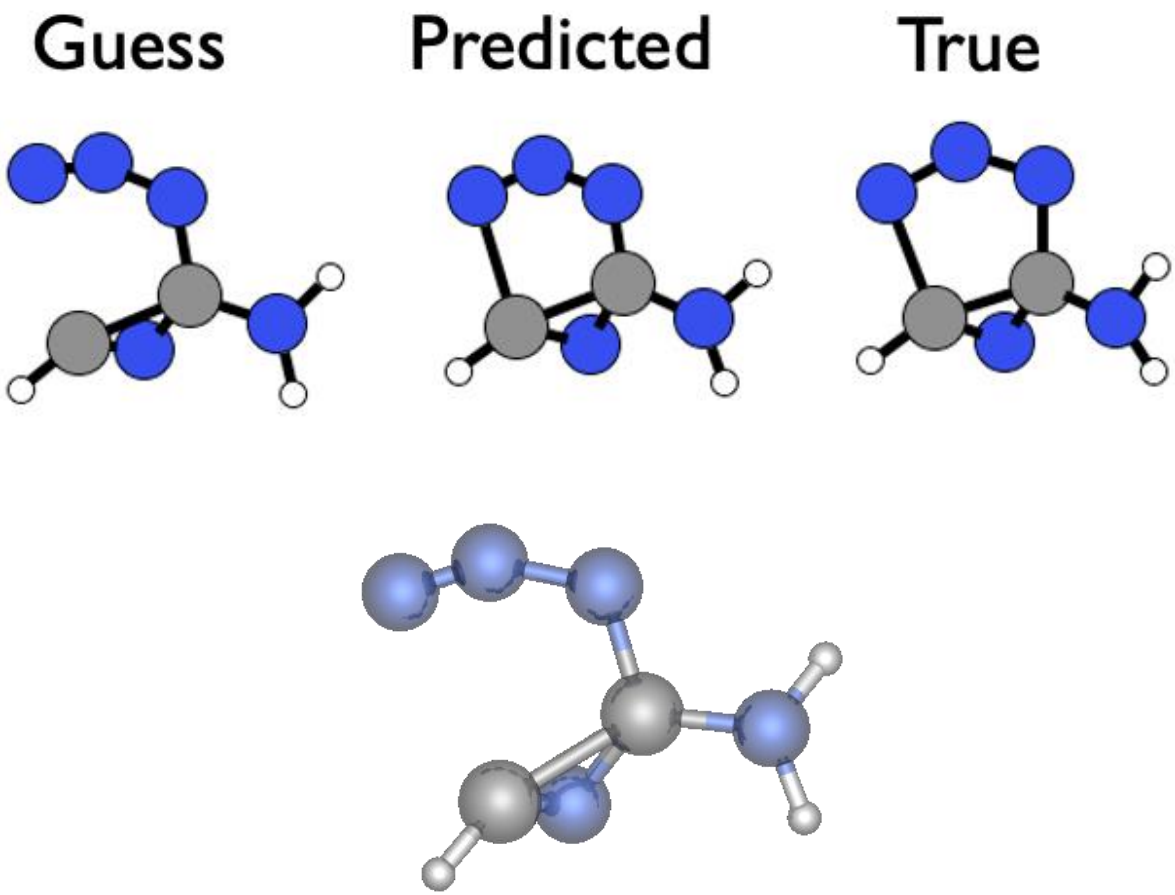


Should be easily loadable on the GPU with a GNN



- Task is to use only R and P to guess TS
- Start with an initial TS guess and learn a function that can get you to the true TS
- Preferred TS guess: midpoint (R+P)/2
- Other TS guesses (xTB, NEB-CI, etc.)
- Refer to `Data/transition1x/train.pkl`
  - Test and validation dataset also included

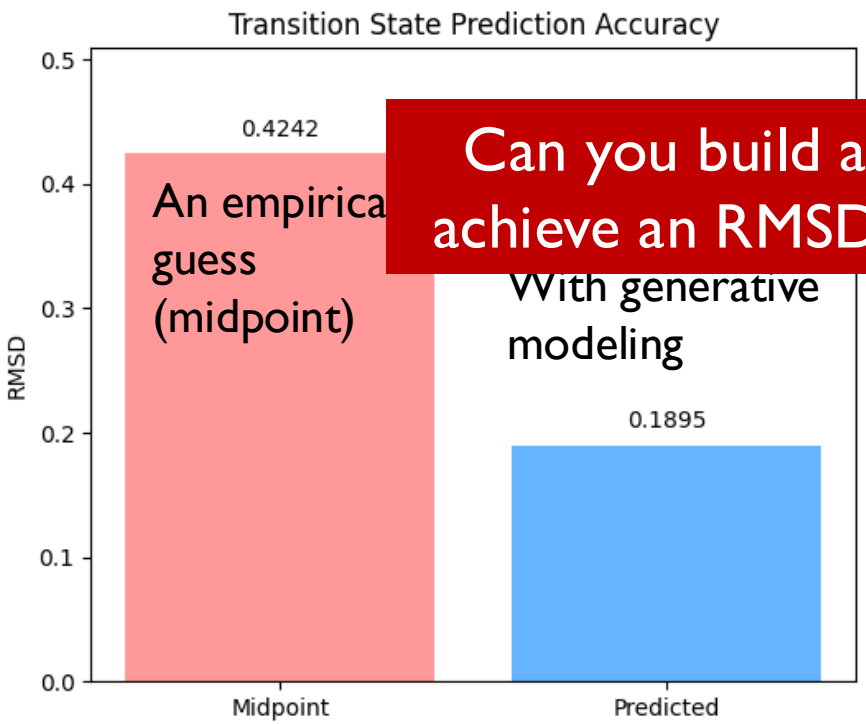
Example of good TS prediction



Evaluation

Root mean squared deviation of predicted TS geometry to true TS geometry

$$\mathrm{RMSD}(\mathbf{v}, \mathbf{w}) = \sqrt{\frac{1}{n}\sum_{i=1}^{n} \|v_i - w_i\|^2}$$
$$= \sqrt{\frac{1}{n}\sum_{i=1}^{n}((v_{ix} - w_{ix})^2 + (v_{iy} - w_{iy})^2 + (v_{iz} - w_{iz})^2)}$$

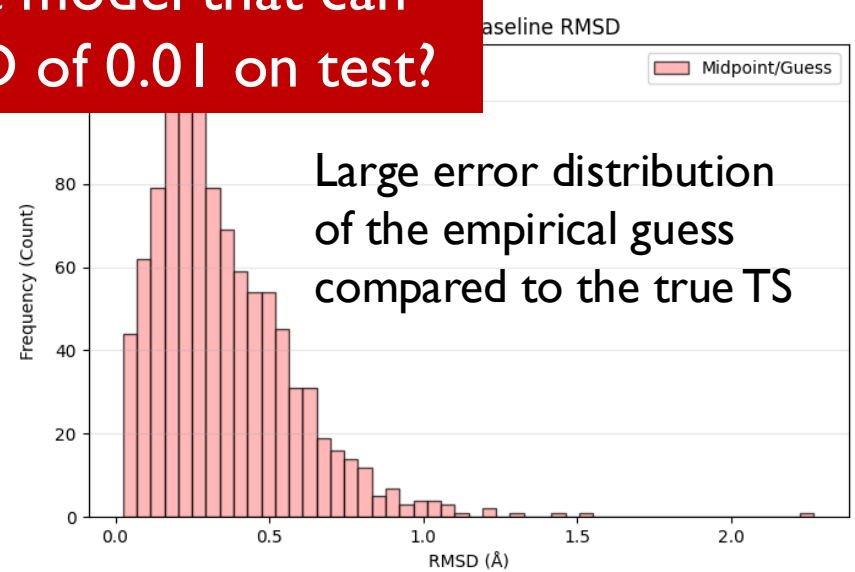


An empirica
guess
(midpoint)

With generative
modeling

Can you build a model that can achieve an RMSD of 0.01 on test?

e to beat



Large error distribution of the empirical guess compared to the true TS

There are papers that report RMSD in the order of 0.01!

Encoding molecular data

Equivariant GNN (EGNN)

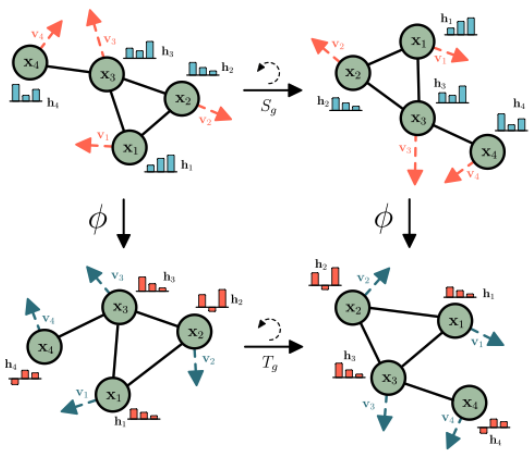


Figure 1. Example of rotation equivariance on a graph with a graph neural network ϕ

Why equivariance matters:

- Physics does not depend on coordinate frame
- The same molecule rotated should give the same prediction (up to rotation)

Neural networks that respect physical symmetries:

- Translation
- Rotation
- Permutation of atoms

$$f(Rx + t) = R\,f(x)$$

- Relative geometry (translation)

$$r_{ij} = x_i - x_j, \qquad d_{ij} = \|x_i - x_j\|^2$$

- Message passing

$$m_{ij} = \phi_m(h_i, h_j, d_{ij})$$

- Node feature update (perm. inv.)

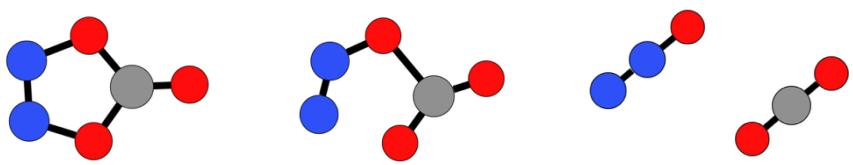
$$h_i^{(l+1)} = h_i^{(l)} + \phi_h\left(\sum_j m_{ij}\right)$$

- Equivariant coordinate update

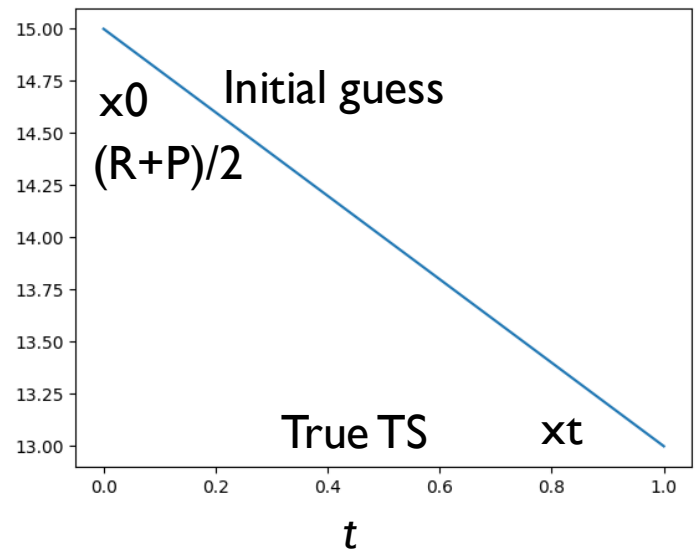
$$x_i^{(l+1)} = x_i^{(l)} + \sum_j (x_i - x_j)\,\phi_x(m_{ij})$$

Example: flow matching: learning how atoms move

- Participants can use any ML approach. The github repo has a flow matching example
- Instead of generating a transition state from scratch, we start from a reasonable guess and learn how atoms should move toward the true transition state.



R TS P



Interpolation path

$$x_t = (1-t)\,x_0 + t\,x_{TS}, \quad t \in [0,1]$$

Learn the velocity field

$$u_\theta(x_t, t \mid c) \qquad c = \{x_R,\ x_P,\ Z\}$$

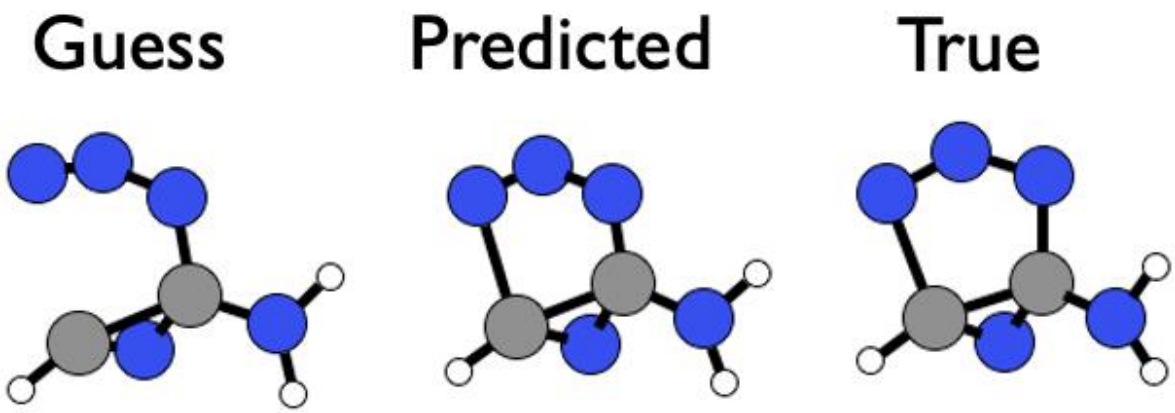
Target velocity field

$$u^*(x_t, t) = \frac{dx_t}{dt} = x_{TS} - x_0$$

Loss (randomly sample along the path)

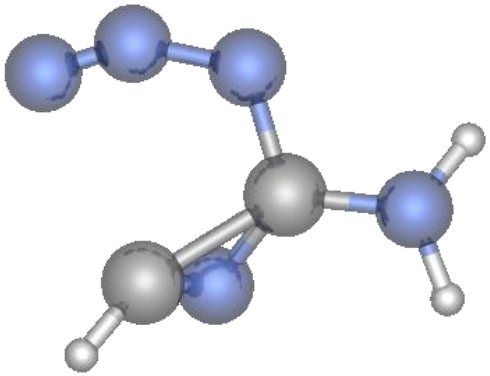
$$\mathcal{L} = \mathbb{E}_{t\sim\mathcal{U}(0,1)}\left[\left\|u_\theta(x_t, t \mid c) - (x_{TS} - x_0)\right\|^2\right]$$

Example of good TS prediction



When you integrate

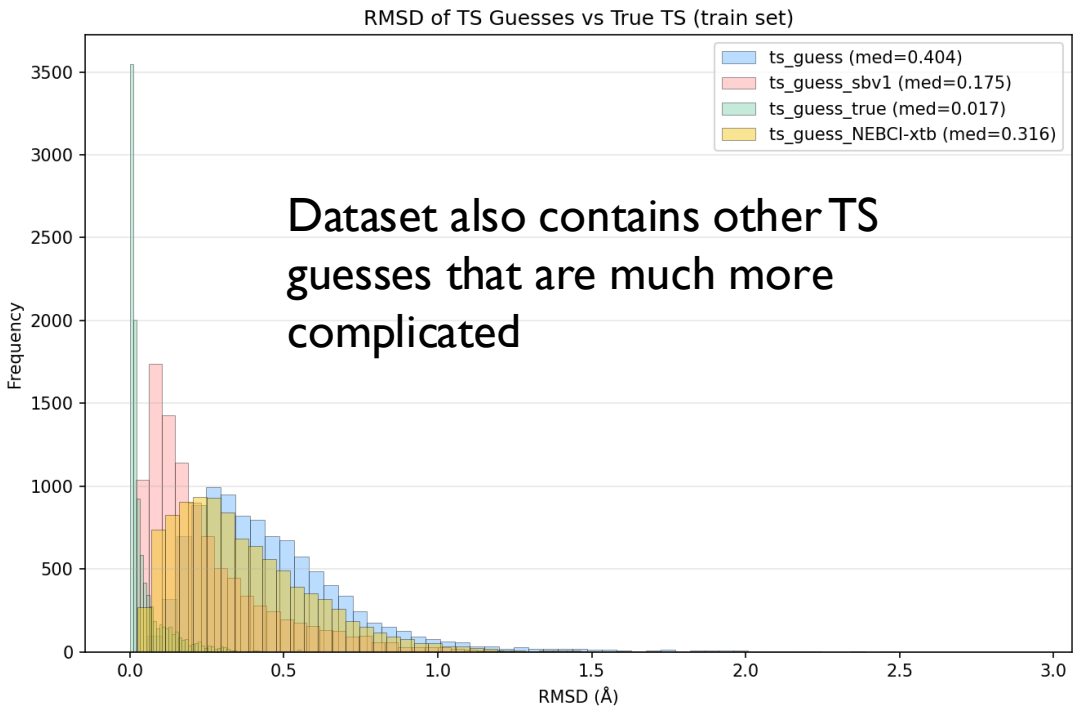
$$\frac{dx}{dt} = u_\theta(x, t, c)$$



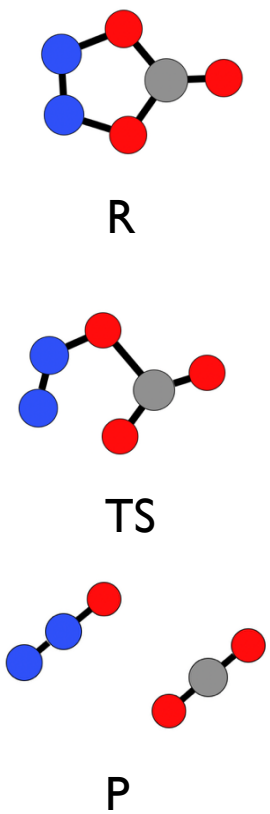
Other baselines

The following information from dataset can also be used:

- Multiple pre-computed TS guesses (there's a penalty)
- Energies and forces (DFT level)



Dataset also contains other TS guesses that are much more complicated



Data set and baseline code

First, understand the dataset

What's there in the transition 1x dataset: (train.pkl, test.pkl, val.pkl)

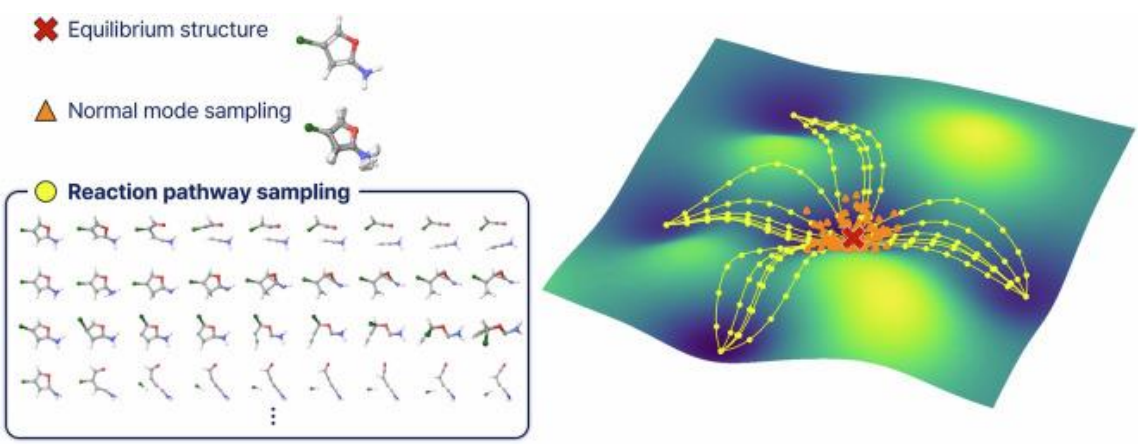
- top-level keys: **reactant, product, transition_state,** single_fragment, use_ind, ts_guess, ts_guess_sbv1, ts_guess_true, ts_guess_NEBCI-xtb
- per-structure keys (inside **reactant/product/transition_state**): positions, charges (or atomic_numbers), num_atoms, fragments, rxn, and optional quantum properties (wB97x_6-31G(d).energy, forces, atomization energy)
- Can refer to https://gitlab.com/matschreiner/Transition1x for additional details of the dataset.
- Refer to the baseline script train_and_eval_egnn.py and example_baseline_reactOT.ipynb
  - Note that the code is simplified to just a fixed atom count. Your final code must be generalized for any type of atom (representation is key)
- You can use pre-built packages for EGNN: https://github.com/lucidrains/egnn-pytorch, you are not restricted to use the EGNN functions that we have in the hackathon repo.

Things to think about

- Go through the Notebooks for assistance on how to get started. The github repo has all detailed needed to start working.
- Alignment before computing RMSD: Kabsch algorithm code given in metrics.py script.
- How to increase transferability?
  - Example in github assumes fixed number of atoms. Transition1x has varying number of atoms
  - Inclusion of masking in your EGNN representation
- How do you encode your connection in your EGNN?
  - fully connected can be expensive and reduce locality for TS prediction
  - Maybe use kNN – other approaches?
- Is the linear interpolation a good guess for flow matching?
- Is flow matching the best approach at all?
- Any improvements to EGNN?

Extra challenge

1. [Easy] Can your model also predict the transition states of the Halo8 dataset?
2. [Not so easy] Halo8 also contains the full reaction pathway from R to P (including TS). Can you modify your method to predict the full trajectory from R to P?
   1. Massive bonus points to teams that can achieve this! Dataset can be found here /project/ai4s-hackathon/ai-sci-hackathon-2026/data/halo8_full_traj.pkl



Predicting the whole reaction pathway

$x(t), \quad t \in [0,1]$

For each reaction: frames are $\{x^{(k)}\}_{k=0}^{K}$

$t_k = \frac{k}{K}$

Supervised samples of the form

$(x(t_k),\ t_k,\ c) \quad c = \{x_R,\ x_P,\ Z\}$

Define conditional velocity field

$u_\theta(x, t \mid c)$

Target velocity field from pathway data

$u^{\backslash *}(x(t_k), t_k) \approx \frac{x^{(k+1)} - x^{(k)}}{t_{k+1} - t_k}$ Finite differences?

Loss using target velocity field

$$\mathcal{L} = \mathbb{E}_{\text{rxn}} \mathbb{E}_k \left[\left\| u_\theta(x^{(k)}, t_k \mid c) - \frac{x^{(k+1)} - x^{(k)}}{\Delta t} \right\|^2 \right]$$

Include other conditions like energy, forces

Inference:

Start from reactant geom

$x(0) = x_R$

Integrate learned ODE

$$\frac{dx}{dt} = u_\theta(x, t \mid c)$$

Sample frames along traj

$x(t_1), x(t_2), \ldots, x(1)$

- Won't be using the linear bridge approximation

Getting started with RCC

- We recommend using the RCC Cluster (Midway3) for model training (with GPUs)
- We encourage local development on your machines for simpler calculations (testing)
- Log into Midway3:

```
ssh -Y [cnetid]@midway3-login3.rcc.uchicago.edu
```

- Do not run heavy calculations like model training on the login node. Submit a job on RCC to submit these calculations
- We have Schmidt GPUs reserved for all participants till April 28th

```
sinteractive --account=ai4s-hackathon --partition=ai4s-hackathon --reservation=ai4s-hackathon --gres=gpu:1
```

- You can run heavy calculations on these nodes. There are additional interactive arguments in here: https://docs.rcc.uchicago.edu/slurm/sinteractive/

Job submissions on RCC

- For batch jobs on GPU nodes, use

```
#SBATCH --account=ai4s-hackathon

#SBATCH --reservation=ai4s-hackathon

#SBATCH --partition=ai4s-hackathon
```

- This would go into your sub.sbatch script. Where you would load your environment, load cuda and your python script

```
source activate ENV_NAME
module load cuda
python train_ex.py
```

- You can submit jobs using sbatch sub.sbatch
- You can track your submitted jobs with squeue -u [cnetid]
- Please refer to the rcc.submit directory for more information about submitting jobs on the cluster.

Environment set up

- Rather than duplicating full environments
- We have created a base environment that you can use /project/ai4s-hackathon/ai-sci-hackathon-2026/hackathon-base
- Refer to the repo README for initial set up
- You can then create your own personal virtual environment in your home directory with: python -m venv --system-site-packages ~/my-hack-venv
    - Install all other packages inside your personal environment

Questions?